

Eman Tarek Mohamed 24p0356

The PathPlanning class is designed to compute a smooth, safe driving path for an autonomous vehicle navigating through cone-defined tracks. It uses the car's current pose and detected cones to calculate a sequence of path points (the midline) that the car should follow.

- Blue cones mark the left boundary.
- Yellow cones mark the right boundary.
- The algorithm ensures the car drives between these cones without crossing or clipping them.

It continues generating a forward-facing trajectory, automatically transitioning from curved motion toward the target to a straight segment after passing it.

The main idea is simple yet effective: the car gradually steers toward a computed target point (based on cone positions). Once it passes that target, it locks its steering direction and moves straight forward. This results in a natural and stable path.

Code Explained

The process consists of five key stages:

Stage 1

```
def __init__(self, car_pose: CarPose, cones: List[Cone]):  
    self.car_pose = car_pose  
    self.cones = cones
```

The planner receives: - car_pose: position (x, y) and orientation yaw. - cones: list of cone objects, each with position (x, y) and color (0 = yellow, 1 = blue).

These inputs define the environment and the car's frame of reference.

Stage 2

Cones are separated by color, and the average position (centroid) of each side is calculated. This smooths out minor sensor noise and gives a stable representation of each boundary.

```
yellow_center = average position of all yellow cones  
blue_center = average position of all blue cones
```

This is critical when cones are unevenly distributed — it allows the planner to compute a reasonable “centerline” even if cones aren't perfectly aligned.

Stage 3

Once both sides are identified, the midpoint between their centroids becomes the target point:

```
if blue_center and yellow_center:
    target_x = (blue_center[0] + yellow_center[0]) / 2
    target_y = (blue_center[1] + yellow_center[1]) / 2
```

If only one color is visible, the opposite boundary is estimated by offsetting the visible side by half the assumed track width (≈ 3.5 m). This way, the planner still defines a central target for the car to steer toward.

Stage 4

The car starts generating points along the path, beginning from its position. It calculates the vector from the current position to the target and adjusts its heading slightly each step:

```
current_dx = (1 - TURN_AGGRESSION) * current_dx + TURN_AGGRESSION * vec_to_target_x
current_dy = (1 - TURN_AGGRESSION) * current_dy + TURN_AGGRESSION * vec_to_target_y
```

Here, $\text{TURN_AGGRESSION} = 0.9$ controls how quickly the car turns toward the target. - A high value = aggressive steering. - A low value = smoother, slower adjustment.

The step size between points is 0.5 m, ensuring a dense and consistent path resolution.

Stage 5

Once the car passes the target, the steering direction locks, and the car continues forward in a straight line.

The condition is determined by a dot product test:

```
dot_product = current_dx * vec_to_target_x + current_dy * vec_to_target_y
if dot_product < 0:
    target_is_passed = True
```

If the dot product becomes negative, it means the target is now behind the car. The planner stops adjusting its heading, ensuring the remainder of the path stays straight.

Parameters

Parameter	Purpose	Value
PATH_LENGTH	Total generated path length	10 m
STEP_SIZE	Distance between path points	0.5 m
TRACK_WIDTH	Estimated distance between cone sides	3.5 m
TURN_AGGRESSION	Steering sharpness factor	0.9

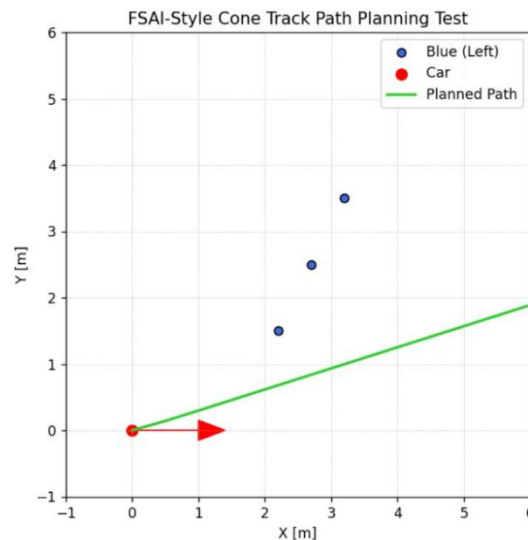
Scenario 21: Blue Cones Only (Left Boundary Visible)

In this scenario, only blue cones are visible, all on the left-hand side of the car. No yellow cones are detected.

Without right-side cones, it's easy for a naive algorithm to incorrectly assume the car should drive close to the left cones or drift outside the lane. so The planner computes the centroid of the blue cones and estimates a virtual right boundary using the known track width. It offsets the visible side laterally by half of $TRACK_WIDTH = 3.5 / 2 = 1.75$ m, ensuring the car stays in the middle of the estimated track.

Result:

- The green planned path passes parallel to the cone line.
- Blue cones stay on the left side.
- The path is correctly aligned with the forward direction of travel.



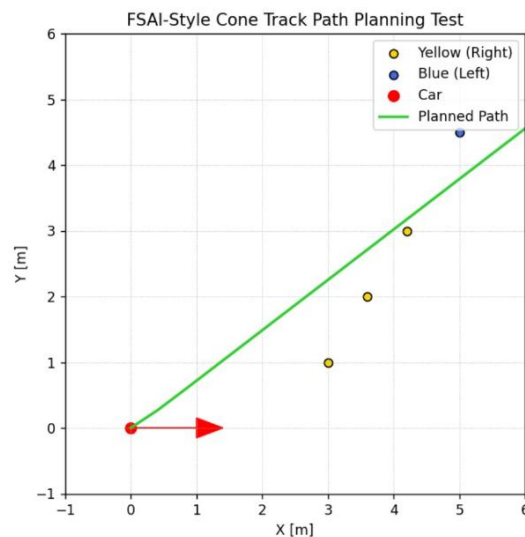
Scenario 22: Full Track with Both Cone Sides

Both blue (left) and yellow (right) cones are present, evenly spaced in a diagonal pattern relative to the car.

The car must steer through the midpoint without cutting too close to either side, especially as the cone arrangement narrows. so Using the centroid method, the algorithm calculates the midpoint between blue and yellow sides. It gradually curves toward this center until it reaches it, then transitions into straight motion once the midpoint is passed (dot product < 0 condition).

Result:

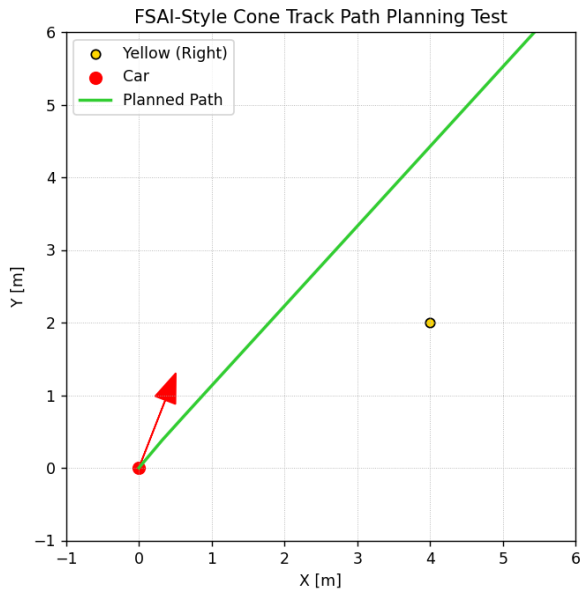
- The generated path is perfectly centered between both colors.
- Blue cones remain to the left and yellow cones to the right throughout.
- The path continues forward, slightly curving at the start and then straightening, reflecting realistic driving dynamics.



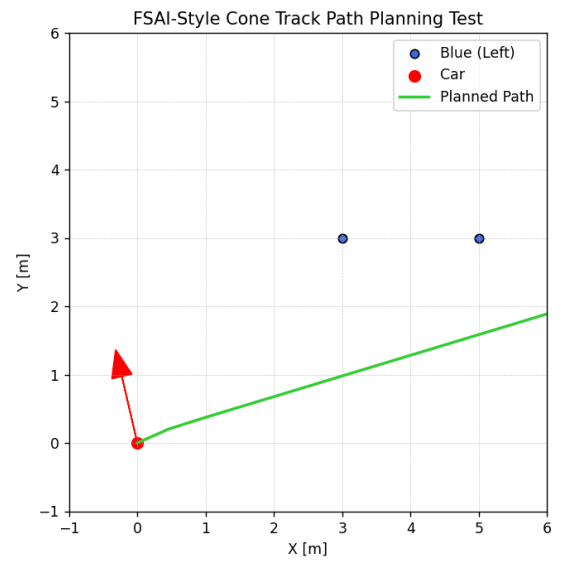
Logic

1. Separate blue and yellow cones
2. Compute centroids of both sides
3. If both $\rightarrow$ take midpoint
If one $\rightarrow$ offset by half track width
4. Generate curved path toward target
5. Once target is behind (dot < 0) $\rightarrow$ go straight

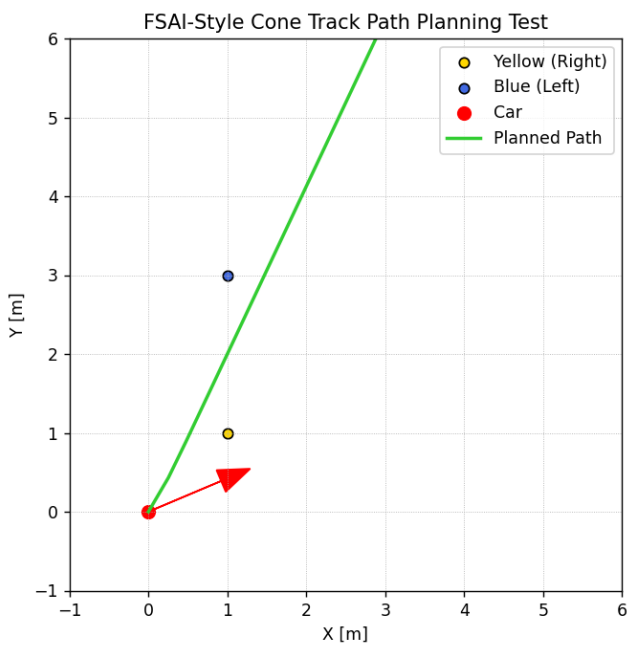
Scenario:



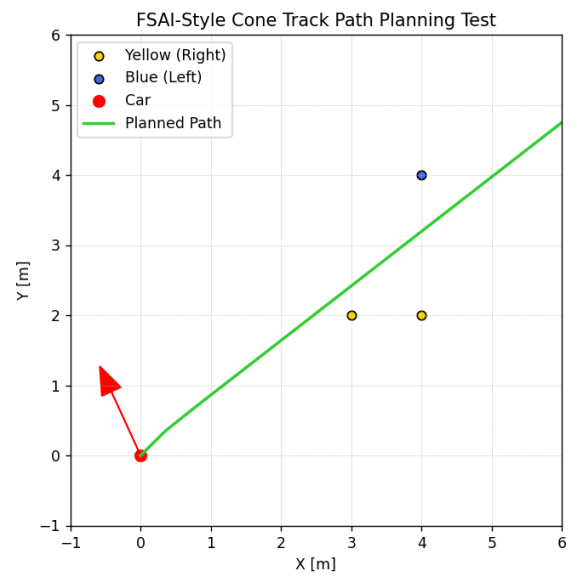
scenario:



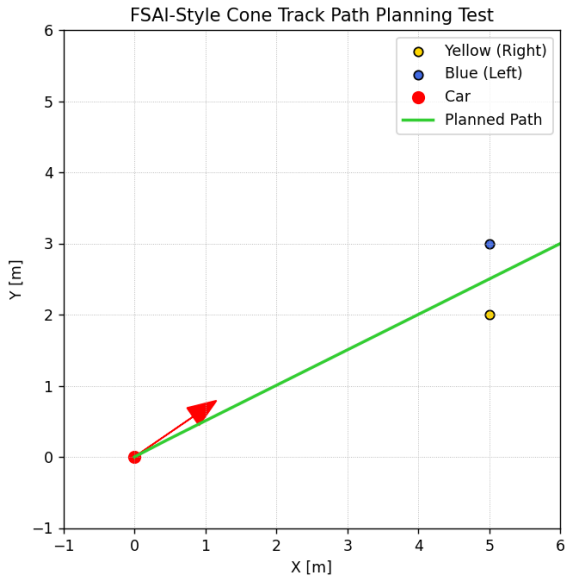
Scenario:



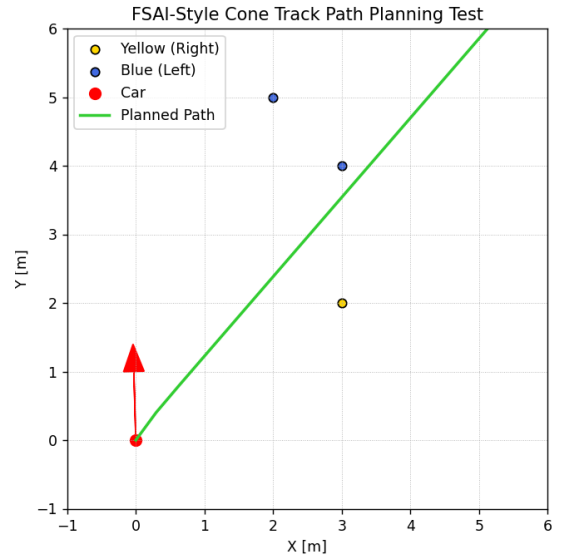
Scenario:



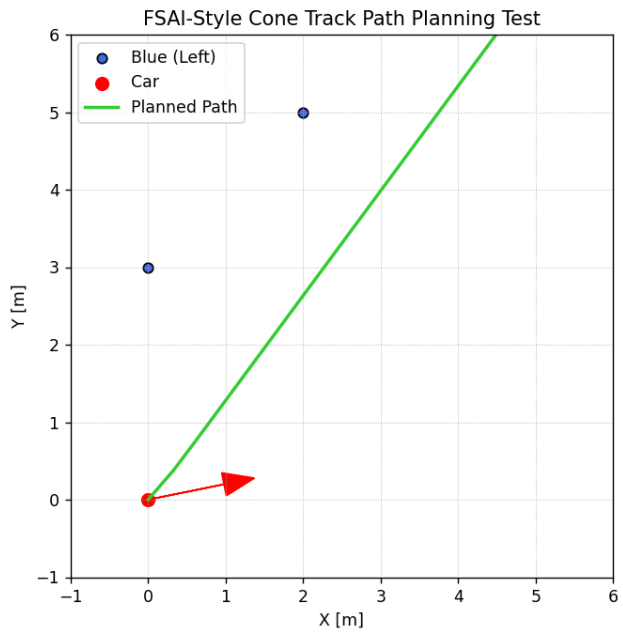
Scenario:



Scenario:



Scenario:



Scenario:

